

STAN: SOFTWARE FOR DATA CURATION

METHODOLOGICAL DESCRIPTION

INTRODUCTION

This document describes the software Stan, named in tribute to UCSF researcher Stanton Glantz and used here as an acronym for System for Tactical Analysis of Documents, developed by André Luiz Oliveira da Silva (ORCID: 0000-0003-4768-959X). The script was designed to automate the querying, filtering, and extraction of a targeted corpus from broader documentary collections, producing a research-ready subset rather than reproducing an entire source database.

Crucially, STAN operates as a strictly offline, decentralized tool that processes text corpora stored locally on the user's infrastructure. It requires the raw target datasets to be downloaded beforehand and saved onto a local storage drive (HDD, SSD, or local network share) before execution, as the system does not perform live cloud queries or remote web scraping. Its main purpose is to serve as a core component of a data-preprocessing pipeline, transforming raw Industry Documents Library (IDL) materials into structured outputs suitable for subsequent use in Artificial Intelligence (AI) and Natural Language Processing (NLP) workflows. In its current implementation, STAN processes structured datasets stored in CSV format, as well as PDF documents, and it can optionally apply OCR through Heimdall Mode to improve text recovery from scanned or low-text materials. The script generates machine-readable, newline-delimited compressed JSON files (JSONL) alongside human-readable reports, enabling both computational ingestion and manual validation for qualitative and methodological assessment.

CORE FUNCTIONALITY AND DESIGN

STAN is implemented as a standalone desktop application using CustomTkinter, a graphical interface layer built on top of Tkinter. This design allows researchers to configure data-processing parameters without editing the source code directly. The workflow is organized around three main phases: parameter specification, corpus processing, and output generation.

USER INTERFACE AND PARAMETERIZATION

The graphical interface enables the researcher to define the parameters necessary to build a reproducible and well-characterized dataset:

- **Target Corpus Location:** The user specifies one or more parent directories containing the raw data files. The script recursively traverses all

subdirectories to locate relevant files (.csv, .pdf, and image formats when Heimdall Mode is active). Because the application relies entirely on local processing, users must populate these directories with the official datasets provided by UCSF, which are permanently available for public download at the university's repository (<https://ucsf.app.box.com/v/IDL-DataSets/folder/78644252849>).

- **Boolean Search Queries:** A text box where users can build complex Boolean queries. Supported syntax includes:
 - **AND:** returns documents where all terms appear.
 - **OR** (comma-separated or using the word 'OR'): returns documents containing any of the listed terms.
 - **NOT:** excludes documents containing specified terms.
 - **Exact phrase** (double quotes): matches an exact string sequence.
 - **Wildcard suffix** (asterisk-based): matches words sharing a prefix (e.g., pharm* matches pharmacy, pharmacist, etc.).
- **Document-Type Filter:** The script reads a predefined set of document types (e.g., Report, Letter, Memorandum) from structured metadata. The user may select one or more types to refine the search.
- **Geographical and Temporal Filters:** Users can refine searches chronologically by setting a specific year range (Start and End dates), with a toggle to include or exclude records marked as 'Unknown Date'. Additionally, a geographical filter allows users to isolate documents that explicitly mention specific countries, dynamically matching the text against a comprehensive list of World Health Organization (WHO) member states.

DETAILED USAGE INSTRUCTIONS

To execute a search and extraction routine using STAN, users should follow this standard workflow directly from the GUI:

1. **Search Terms:** Enter target keywords in the main search bar using advanced Boolean operators to refine retrieval.
2. **Select Folders:** Click 'Select Folders' to choose the local root directory containing your downloaded UCSF datasets. The script will automatically and recursively scan all subdirectories within the path.
3. **Apply Filters (Optional):**

- Year Filter: Input a 'Start' and 'End' year to restrict the chronological scope. Checking the 'Incl. Unknown' box ensures that documents missing explicit date metadata are retained in the results.
 - Document Type: Select desired document classifications based on the archive's native categories (e.g., reports, letters, memos).
 - Country: Filter records based on specific geographical mentions of WHO member states.
4. OCR Configuration (Optional): Check 'Heimdall mode (OCR)' to forcefully extract text from image files and scanned PDFs. Note: Enabling this feature significantly increases total processing time.
 5. Execution: Click the 'START SEARCH' button. The system will prompt for a destination folder where the final reports and JSON files will be stored.
 6. Post-Processing & Packaging: Once the search is successfully completed and system memory is freed, the user can utilize the packaging options:
 -  Build AI Packs (JSON/MD)': Automatically compresses and stitches the exported JSON and Markdown files into token-optimized blocks ready for AI platforms like NotebookLM or ChatGPT.
 -  Concat HTML': Merges all individual HTML reports into continuous browsing pages (up to 100MB each) for smooth browser rendering without crashing.
 7. Generate Timeline: At any time, users can click the "Generate Timeline" button to select an exported JSONL file. The system will read the chronological metadata and output a PNG line chart illustrating the document volume over time.

DATA INGESTION AND PROCESSING MECHANISM

Once initiated, STAN processes the selected files according to their specific formats and applies a unified filtering logic across the corpus.

- CSV File Processing: The script leverages pandas for fast database reading. Legacy UCSF CSV files are read in chunks to prevent memory overload during processing.
- Parallel Processing Architecture: To efficiently handle large textual corpora and computationally expensive OCR tasks, STAN utilizes a concurrent execution model based on ProcessPoolExecutor, distributing reading, regex

matching, and OCR processing of multiple files simultaneously across available CPU cores.

- **PDF File Processing:** The script employs the PyMuPDF (fitz) library to extract text page by page. If the recovered text matches the Boolean query criteria, a documentary record is structured identically to the table-based results.
- **Image and Scanned Document Processing (Heimdall Mode):** To handle non-searchable PDFs and scanned image formats (.jpg, .jpeg, .png, .tiff), STAN integrates the "Heimdall Mode" OCR module. When enabled, it utilizes a portable, localized Tesseract OCR engine operating in tandem with the Python Imaging Library (PIL) to extract embedded text across multiple languages, including English, Portuguese, and Simplified Chinese, ensuring the indexing of scanned or deliberately obfuscated documents. OCR is selectively applied to PDF pages where native text extraction proves insufficient.

OUTPUT GENERATION

STAN generates structured outputs for machine processing and formatted reports for human analysis, all controlled by timestamps.

- **Main Output (Machine-Consumable):** One or more compressed JSON Lines (.jsonl.gz) files. To eliminate memory overflow risks and ensure strict compatibility with Large Language Model (LLM) context windows, the script incorporates a dual-trigger batching mechanism: records are flushed from RAM to the hard drive as soon as the batch reaches a 45 MB physical limit OR a 400,000-word semantic ceiling, whichever occurs first. This results in highly efficient, perfectly sized compressed files for AI tools.
- **AI-Optimized Markdown Exports:** The script automatically generates structured Markdown (.md) files specifically tailored for ingestion in Retrieval-Augmented Generation (RAG) applications, such as Google NotebookLM. Governed by the same dual-trigger safety mechanism, these files preserve essential hierarchical formatting for semantic parsing without breaching AI token quotas.
- **Human-Readable Reports & Zotero Reference Management:** In parallel, detailed reports are generated in .txt and .html formats. The HTML report is fully self-contained, paginated, and interactive, including direct hyperlinks pointing to the source items on the official UCSF Industry Documents Library website. It features an integrated "sticky" export panel allowing researchers to select desired documents via checkboxes and instantly export their rich

metadata into a standard .RIS file, establishing a direct bridge with reference managers like Zotero.

- **Timeline Chart Visualization:** A visual PNG chart can be natively generated to quickly identify historical peaks and trends within the extracted corpus based on the document dates.

DECOUPLED POST-PROCESSING ARCHITECTURE

To ensure hardware stability during massive extractions, STAN separates final consolidation into decoupled post-processing modules. After the parallel extraction phase concludes and CPU/RAM resources are fully released by the garbage collector, the user can trigger two distinct tools:

1. **AI Packs Engine:** This engine uses the tiktoken library for exact GPT-4 model token counting, bundling the JSONL and Markdown fragments into consolidated super-packages of up to 2,000,000 tokens in an isolated folder (AI_Ready_Packages), ready for immediate AI use.
2. **HTML Stitching Engine:** To bypass browser memory limitations, this module merges individual HTML outputs into continuous reading pages explicitly capped at dynamic 100MB limits. It uses intelligent chunking to prevent documents from being cut in half, maintaining CSS and layout integrity.

WHY ARE STAN'S RESULTS DIFFERENT FROM THOSE FOUND ON THE UCSF TRUTH INDUSTRY DOCUMENTS PORTAL?

When cross-referencing searches performed in STAN with the UCSF Industry Documents Library online portal, it is common and perfectly normal to observe a difference in the total number of returned documents. This variation does not indicate a flaw in the application's search engine; rather, it reflects profound architectural differences between a web platform designed for maximum discovery and a data mining tool focused on analytical precision. While the UCSF website acts as a broad "dragnet," STAN is designed to operate as a scalpel, prioritizing strict textual relevance over raw volume.

- **Search Scope (Text vs. Metadata):** The first major cause of this discrepancy lies in the search scope. The UCSF web engine simultaneously scans the content of the documents and all of their associated metadata, such as the title, author, archivist notes, and collections. STAN, by contrast, is programmed to focus its analysis specifically on the core content of the document, the title and the OCR text body. This means that if your search

term appears in an obscure catalog note or an author field, but is not present in the actual title or the text read from the page, STAN will deliberately exclude it from the results. This approach ensures that the context exported for artificial intelligence contains the exact empirical evidence requested.

- **Matching Rigidity (Strict Regex vs. Semantic Search):** Another determining factor is the rigidity of word matching. UCSF uses semantic search algorithms and fuzzy matching, which can automatically find plural variations, verb roots, or forgive minor scanner reading errors (e.g., returning "nicotine" when the digitized text reads "nicot1ne"). To avoid false positives that could contaminate an academic study, STAN uses Regular Expressions (Regex) with strict word boundaries (\b). The script looks exactly for the character string the user typed. To emulate a broader search in STAN, the user must purposefully resort to the wildcard operator, using an asterisk-based suffix (e.g., nicotin*).
- **Static Database and Hardware Protections:** Finally, there are differences related to the static nature of the database and hardware protections. Unlike the live UCSF repository, which is updated and corrected daily, STAN operates as a strictly offline, decentralized tool that processes text corpora stored locally on the user's infrastructure. It works with a "snapshot" of the data at the moment the user downloaded the raw target datasets. Furthermore, when STAN processes individual PDF or image files on the local machine, it triggers a safety limit to prevent RAM overload, restricting the OCR text reading to the first 50,000 characters of each document.

WHY THE NAME "HEIMDALL"

The name Heimdall derives from Norse mythology, in which Heimdall is portrayed as the vigilant guardian of the gods, endowed with extraordinary perception, especially exceptional sight and hearing. Heimdall is also capable of perceiving across different realms. The name is therefore especially apt for this function in STAN, since Heimdall Mode is intended not only to detect text that conventional extraction methods may overlook, but also to extend the script's reach across different file types and document forms. In methodological terms, the function enhances textual visibility across heterogeneous materials, making Heimdall an appropriate metaphor for cross-representational data detection.